Binary CAN Intrusion Detection Using Engineered Message Features and a Neural Network Baseline

Terence Mpofu, Einstein Gonzo, Tinotenda Taderera

***Abstract*—Controller Area Network (CAN) remains the core communication backbone in many modern vehicles, but its lack of built-in authentication and encryption makes it vulnerable to cyberattacks such as spoofing and denial-of-service. This project investigates binary intrusion detection for CAN traffic using a neural-network baseline trained on processed message-level features. The implementation uses engineered features derived from CAN frames, including CAN ID encoding, payload byte values, inter-arrival timing, and payload summary statistics. A multilayer perceptron (MLP) was trained for normal-versus-attack classification and achieved strong test performance, with 98.00% accuracy, 98.35% precision, 99.55% recall, 98.94% F1-score, and 0.9627 ROC-AUC at a threshold of 0.90. A preliminary multiclass experiment was also conducted but produced weaker results, indicating that attack-type discrimination is more difficult than broad anomaly detection. These results show that the current binary pipeline is stable, reproducible, and effective for CAN intrusion detection.**

***Index Terms*—CAN bus, intrusion detection, automotive cybersecurity, neural network, binary classification, anomaly detection**

I. Introduction

Modern vehicles rely heavily on the Controller Area Network (CAN) protocol to support communication among electronic control units (ECUs). Although CAN is efficient and lightweight, it was designed without strong security protections such as sender authentication or message encryption. As a result, malicious actors who gain access to the bus may inject spoofed messages, flood the network, or manipulate critical vehicle behavior.

This project addresses the problem of CAN intrusion detection using machine learning. The original proposal considered both binary detection and a possible extension to multiclass attack classification. For the checkpoint implementation, the primary focus is binary classification: distinguishing normal CAN traffic from malicious traffic. This focus is motivated by the fact that reliable attack detection is the most immediate practical requirement in an automotive intrusion detection setting.

The main contribution of this checkpoint is a complete and reproducible binary learning pipeline built from processed CAN traffic features. The implementation includes data loading, feature preparation, missing-value handling, train/validation splitting, model training, and evaluation. Initial results show that a simple neural baseline can achieve strong performance on the binary task.

II. Related Work

The security weaknesses of CAN are well documented. The protocol was designed for efficiency and reliability, but not for authentication or confidentiality, which allows attackers with bus access to inject, replay, or suppress messages [1]. Real-world demonstrations, including the remote compromise of a Jeep Cherokee, showed that CAN manipulation can affect safety-critical vehicle functions and helped motivate practical in-vehicle intrusion detection research [2].

Early CAN IDS methods were largely statistical or rule-based. These approaches monitor message frequency, inter-arrival timing, identifier entropy, or payload consistency and are attractive because they are lightweight enough for embedded automotive environments [3], [5]. However, they often depend on hand-crafted thresholds and may be less effective against attacks that closely mimic legitimate traffic. Surveys of the field note that these methods remain important baselines, especially when real-time deployment cost is a major constraint [4], [10].

Machine learning and deep learning methods were introduced to improve flexibility and detection accuracy. Supervised models such as support vector machines, decision trees, and k-nearest neighbors can perform well when CAN traffic is represented with informative structured features [6]. Deep models further reduce manual feature design by learning nonlinear patterns directly from CAN sequences or payloads. Prior studies have explored deep neural networks [7], GAN-based detection [8], and LSTM autoencoder architectures such as CANnolo for anomaly detection [9]. These results suggest that sequence-aware models can capture temporal behavior that simpler baselines may miss, but they also increase computational cost and data requirements.

Recent literature also emphasizes two persistent challenges: dataset quality and model generalization. Public CAN IDS datasets have historically differed in fidelity, attack realism, and labeling, making direct comparison difficult [10]. Newer resources such as can-train-and-test were proposed specifically to evaluate cross-vehicle and cross-attack generalization [11]. This motivates our current design choice: we first establish a stable binary baseline on engineered message-level features before extending to more complex temporal or explainable models. That strategy is consistent with the broader literature, which shows that strong lightweight baselines remain valuable in automotive IDS research [3], [4].

III. Methodology

A. Problem Formulation

The primary task is binary classification:

- Class 0: normal CAN traffic

- Class 1: malicious CAN traffic

A preliminary multiclass formulation was also explored for distinguishing specific attack categories, but the binary task remains the main focus of this checkpoint.

B. Dataset Structure

The implemented dataset is a processed tabular CAN dataset in which each row represents a CAN message and each column represents either a raw field, an engineered feature, metadata, or a label. The binary dataset contained the following columns:

- timestamp, dlc, direction
- class_label, source_file, binary_label
- can_id_int
- byte_0_int through byte_7_int
- inter_arrival
- payload_sum, nonzero_bytes, payload_unique_values
- binary_target

For the binary model, the target variable was binary_target. The columns binary_label, class_label, source_file, direction, and timestamp were dropped from the final baseline to reduce instability, metadata leakage, and unnecessary complexity.

C. Feature Engineering and Cleaning

The model used 14 final input features:

- dlc
- can_id_int
- byte_0_int to byte_7_int
- inter_arrival
- payload_sum
- nonzero_bytes
- payload_unique_values

The inter_arrival feature was computed from timestamp differences within each source file. Because this operation introduces missing values for the first message in each sequence, the preprocessing pipeline explicitly handled missing and invalid values. Infinite values were replaced with missing values, numeric columns were filled using median imputation, and any remaining missing values were filled with zero. Features were then standardized using StandardScaler.

D. Train/Validation Setup

The checkpoint implementation used an 85/15 train/validation split with stratification to preserve class balance. Class weights were computed from the training labels to reduce the effect of class imbalance.

The final training and validation shapes for the binary model were:

- Train: 14,924,594 samples, 14 features
- Validation: 2,633,752 samples, 14 features

E. Model Architecture

The binary classifier used in this study is a multilayer perceptron (MLP) designed for structured CAN bus data. The model takes engineered message-level features as input and learns nonlinear patterns that distinguish normal CAN traffic from malicious traffic. The final input to the network consists of 14 numeric features.

The implemented MLP contains three fully connected hidden layers with ReLU activation, followed by a sigmoid output unit for binary classification. The architecture is summarised as follows:

- Input layer: 14 engineered CAN features
- Dense(64, ReLU)
- Dropout(0.3)
- Dense(32, ReLU)
- Dropout(0.2)
- Dense(16, ReLU)
- Dense(1, sigmoid)

The first hidden layer learns initial relationships among the engineered CAN features, such as payload structure, message size, identifier patterns, and timing behavior. The second hidden layer captures deeper interactions between these learned representations, allowing the network to model more complex distinctions between normal and malicious traffic. Dropout layers were included for regularization, reducing overfitting by randomly deactivating a fraction of neurons during training. The third hidden layer learns more abstract patterns associated with attack behavior before passing the learned representation to the output layer.

The final output layer contains a single neuron with sigmoid activation, which produces a probability between 0 and 1. This probability represents the model's confidence that a given CAN message belongs to the attack class. A classification threshold is then applied to convert the output probability into the final binary decision: normal or attack.

Fig. 1 illustrates the overall MLP architecture used for binary CAN intrusion detection.

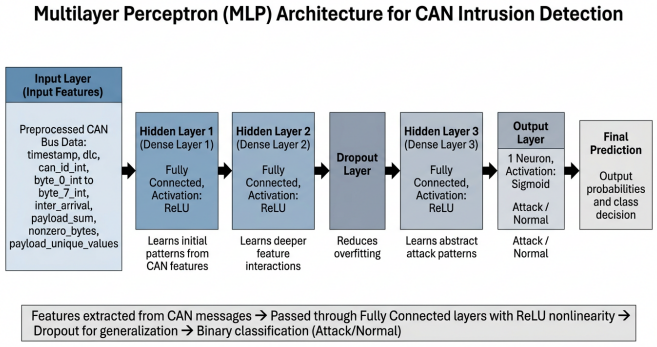

Fig. 1. Multilayer perceptron (MLP) architecture used for binary CAN intrusion detection. Engineered CAN message features are passed through fully connected hidden layers with ReLU activation and dropout regularization, followed by a sigmoid output layer for attack-versus-normal classification.

This architecture was selected because the dataset is tabular and feature-based rather than image- or sequence-based in its final training form. For this type of structured input, an MLP provides a simple and effective baseline that can learn nonlinear decision boundaries without requiring the higher computational cost of more complex sequential models. Al-

though the architecture is relatively lightweight, the results show that it is strong enough to capture important attack-related patterns in CAN traffic.

IV. Experimental Results

A. Binary Test Performance

The final binary model was evaluated on the held-out test set using a decision threshold of 0.90. The model achieved strong overall classification performance, with especially high recall and precision for the attack class. Table I summarizes the main evaluation metrics.

TABLE I
Binary Test Results (Threshold = 0.90)

Metric	Value
Accuracy	0.9800
Precision	0.9835
Recall	0.9955
F1-score	0.9894
ROC-AUC	0.9627
Balanced Accuracy	0.8579
Specificity	0.7203

These results show that the model is highly effective at identifying malicious CAN traffic. In particular, the recall of 0.9955 indicates that the model misses very few attack instances, which is desirable in an intrusion detection setting. Precision is also very high at 0.9835, meaning that most messages flagged as attacks are truly malicious. Although overall accuracy is high, the lower specificity of 0.7203 shows that distinguishing normal traffic from attack traffic remains more challenging than detecting attacks themselves.

B. Confusion Matrix Analysis

The confusion matrix for the test set is shown in Fig. 2. The model correctly classified 2,474,126 attack messages and 106,840 normal messages. However, it also produced 41,491 false positives and 11,295 false negatives.

The confusion matrix confirms that the classifier strongly favors detecting the positive class, which is appropriate for security monitoring where missed attacks are often more costly than false alarms. The relatively small number of false negatives supports the very high recall value. At the same time, the false positive count explains the lower specificity and suggests that further threshold tuning or additional feature refinement may help improve normal-traffic discrimination.

C. ROC and Precision–Recall Analysis

To evaluate ranking performance independently of the fixed classification threshold, ROC and Precision–Recall curves were also generated. The ROC curve in Fig. 3 shows a test ROC-AUC of 0.9627, indicating strong separability between normal and malicious traffic.

The Precision–Recall curve in Fig. 4 further shows that precision remains very high across a wide range of recall values. This is especially useful in an imbalanced intrusion detection setting, where attack detection quality is often more informative than accuracy alone.

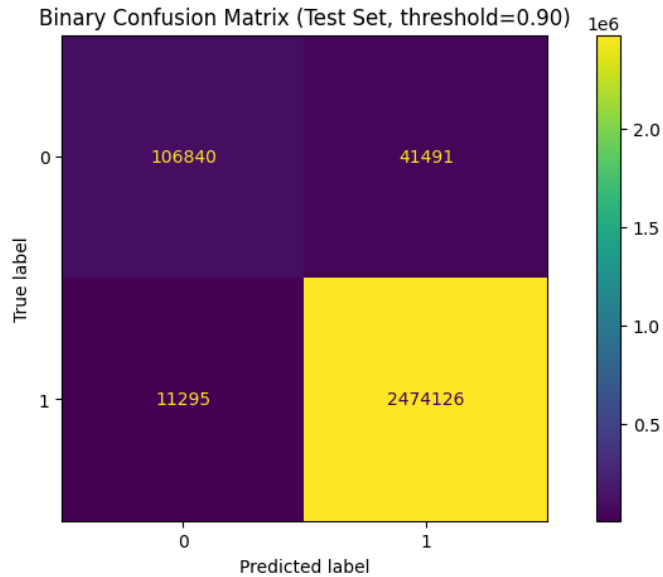

Fig. 2. Confusion matrix for the binary test set at threshold 0.90.

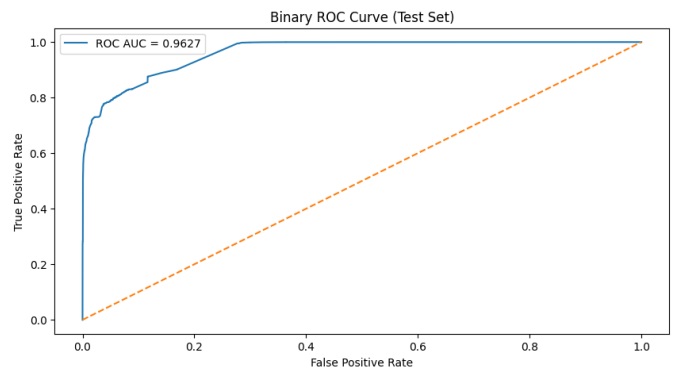

Fig. 3. ROC curve for the binary test set.

Together, these curves indicate that the model learned a strong decision boundary and maintains reliable detection performance beyond a single operating threshold.

D. Training Dynamics

Training and validation curves for loss, accuracy, AUC, precision, and recall are shown in Figs. 5–9. These plots show that the model converged smoothly and remained stable throughout training.

The loss curves decrease steadily without instability, showing that the optimization process was well behaved. Accuracy and AUC rise quickly during the early epochs and then level off, suggesting that the model captures the main discriminative structure of the data relatively early. Precision and recall also remain consistently high throughout training. In several plots, validation performance is slightly higher than training performance, which is not necessarily a sign of error and can

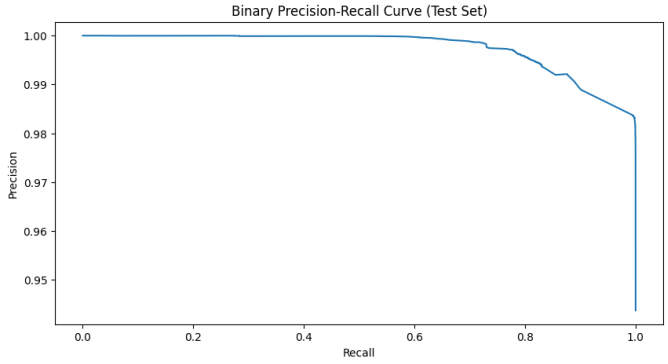

Fig. 4. Precision–Recall curve for the binary test set.

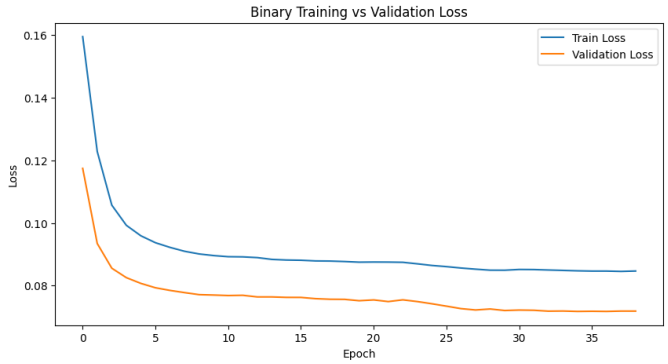

Fig. 5. Training and validation loss for the binary model.

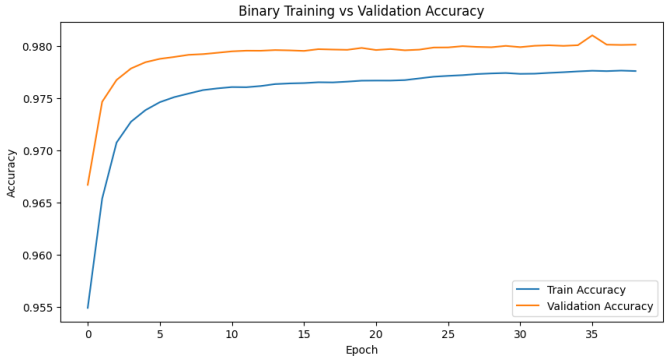

Fig. 6. Training and validation accuracy for the binary model.

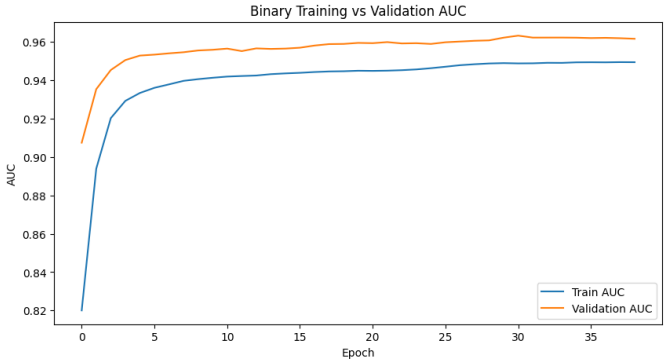

Fig. 7. Training and validation AUC for the binary model.

occur because dropout is active during training but not during validation. Overall, the curves do not show strong evidence of harmful overfitting.

E. Interpretation of Results

Overall, the binary MLP baseline performs very well for CAN intrusion detection. The model is particularly strong at identifying attack traffic, as shown by its very high recall, F1-score, and ROC-AUC. This makes it a useful baseline for safety-critical intrusion detection, where failing to detect malicious activity is costly.

However, the specificity result indicates that the model still misclassifies a nontrivial portion of normal traffic as attacks. This means that while the detector is highly sensitive, there is still room to improve false alarm control. Future improvements may include threshold tuning for different operating requirements, richer temporal features, sequence-based architectures, or calibration techniques to better balance attack detection and normal-message recognition.

F. Preliminary Multiclass Observation

A preliminary multiclass experiment was also explored, but its performance remained weaker than the binary model. This suggests that identifying specific attack categories is a more difficult task than broad normal-versus-attack detection. At this stage, the binary pipeline is the most reliable and best-performing component of the project.

V. Discussion

The strongest result at this stage is the binary classifier. The model demonstrates that even a relatively simple MLP can achieve strong performance when trained on carefully prepared CAN features. The very high precision and recall are especially attractive for intrusion detection because they indicate that the model is both reliable in its attack predictions and highly sensitive to malicious traffic. At the same time, the specificity result shows that some normal messages are still incorrectly flagged as attacks, leaving room for threshold tuning, feature refinement, or temporal modeling.

The multiclass experiment also provides useful insight. While it was not the focus of this checkpoint, its weaker performance suggests that attack-type identification may require richer sequence-aware representations. This supports the proposal recommendation to later explore temporal modeling, such as an LSTM-based extension, once the binary baseline is fully documented.

VI. Conclusion

We established a complete binary CAN intrusion detection pipeline using engineered message-level features and an MLP neural-network baseline. After cleaning missing and invalid values and selecting a stable set of 14 numeric features, the model achieved 98.00% test accuracy, 98.35% precision, 99.55% recall, 98.94% F1-score, and 0.9627 ROC-AUC at a

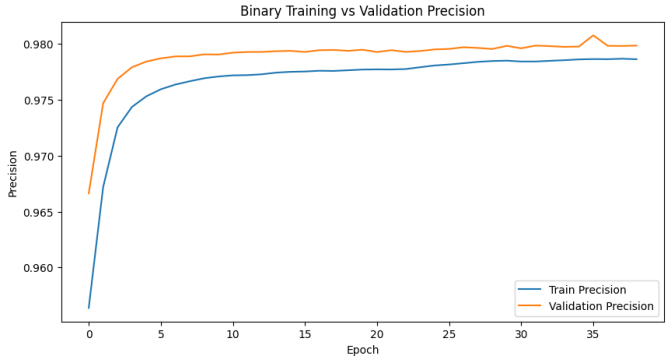

Fig. 8. Training and validation precision for the binary model.

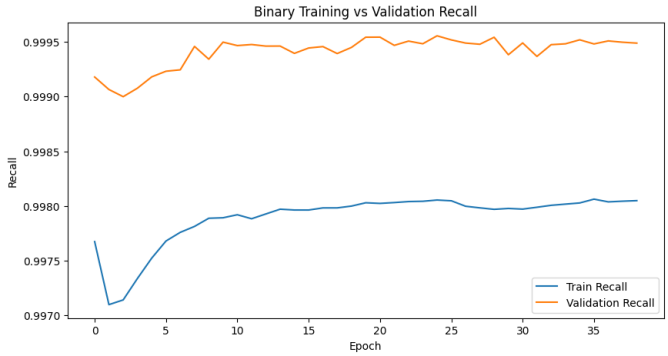

Fig. 9. Training and validation recall for the binary model.

classification threshold of 0.90. These results show that the model is highly effective at detecting malicious CAN traffic and provides a strong baseline for the project.

Although attack detection performance is excellent, specificity remains lower than recall, indicating that some normal messages are still incorrectly flagged as malicious. This leaves room for future improvements through threshold tuning, richer feature engineering, temporal modeling, and calibration. For the final stage of the project, the next steps will be to improve documentation and reproducibility, compare binary and multiclass settings more thoroughly, and explore more advanced architectures for better attack-type discrimination.

References

[1] R. Bosch GmbH, "CAN Specification Version 2.0," Sept. 1991.

[2] C. Miller and C. Valasek, "Remote Exploitation of an Unaltered Passenger Vehicle," Black Hat USA, 2015.

[3] S.-F. Lokman, A. T. Othman, and M.-H. Abu-Bakar, "Intrusion detection system for automotive Controller Area Network (CAN) bus system: a review," *EURASIP Journal on Wireless Communications and Networking*, vol. 2019, no. 184, 2019.

[4] W. Wu, R. Li, G. Xie, J. An, Y. Bai, J. Zhou, and K. Li, "A Survey of Intrusion Detection for In-Vehicle Networks," *IEEE Transactions on Intelligent Transportation Systems*, vol. 21, no. 3, pp. 919–933, 2020.

[5] M. Marchetti, D. Stabili, A. Guido, and M. Colajanni, "Evaluation of anomaly detection for in-vehicle networks through information-theoretic algorithms," in *Proc. IEEE RTSI*, 2016, pp. 1–6.

[6] B. S. Bari, K. Yelamarthi, and S. Ghafoor, "Intrusion Detection in Vehicle Controller Area Network (CAN) Bus Using Machine Learning: A Comparative Performance Study," *Sensors*, vol. 23, no. 7, p. 3610, 2023.

[7] M.-J. Kang and J.-W. Kang, "Intrusion Detection System Using Deep Neural Network for In-Vehicle Network Security," *PLOS ONE*, vol. 11, no. 6, e0155781, 2016.

[8] E. Seo, H. M. Song, and H. K. Kim, "GIDS: GAN based Intrusion Detection System for In-Vehicle Network," in *Proc. 16th Annual Conference on Privacy, Security and Trust (PST)*, 2018, pp. 1–6.

[9] S. Longari, D. H. Nova Valcarcel, M. Zago, M. Carminati, and S. Zanero, "CANnolo: An Anomaly Detection System Based on LSTM Autoencoders for Controller Area Network," *IEEE Transactions on Network and Service Management*, vol. 18, no. 2, pp. 1913–1924, 2021.

[10] M. E. Verma, R. A. Bridges, M. D. Iannacone, S. C. Hollifield, P. Moriano, S. C. Hespeler, et al., "A comprehensive guide to CAN IDS data and introduction of the ROAD dataset," *PLOS ONE*, vol. 19, no. 1, e0296879, 2024.

[11] B. Lampe and W. Meng, "can-train-and-test: A curated CAN dataset for automotive intrusion detection," *Computers & Security*, vol. 140, p. 103777, 2024.